

claude-windows / bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f\subagents\workflows\wf_b8703ccc-78b\agent-a6e21c1a621fa978f.jsonl`  
- SHA-256: `26f6d5bc948862225423cc065670e5de5d9acadcb612d5979d481b02f23a51f4`  
- Source modified: `2026-06-17T14:34:16+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_b8703ccc-78b`  
- session_id: `bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f`
```

Transcript

1. user (2026-06-17T14:33:01.592Z)

Review the diff of undefined/tools/generate_highlights.py vs origin/master (run `git -C undefined diff origin/master...HEAD -- tools/generate_highlights.py`). LENS: BEHAVIOR PRESERVATION. The critical invariant: when `detect_from_paths` is `None` (the default / legacy path), the runner must behave EXACTLY as before ? same `localize`, same `video_id` source, same `stitch` source, same `reel` name, same `cleanup`. Trace every renamed variable (`proc_src` -> `proc_detect`, `staged_src` `basename`) and confirm the `None` path is byte-equivalent in behavior to the pre-diff code. Read the full file and `tests/test_generate_highlights.py` for context. Report any REGRESSION.

2. assistant (2026-06-17T14:33:03.690Z)

I'll analyze the diff to check for behavior preservation in the `generate_highlights.py` file. Let me start by examining the current state and comparing with the original.

3. user (2026-06-17T14:33:04.031Z)

(Bash completed with no output)

4. assistant (2026-06-17T14:33:05.468Z)

Let me check if there's actually a diff. First, let me see the current branch status:

5. user (2026-06-17T14:33:06.213Z)

On branch `docs/golden-regression-fixtures-plan`
Your branch and `'origin/docs/golden-regression-fixtures-plan'` have diverged,
and have 18 and 1 different commits each, respectively.
(use `"git pull"` if you want to integrate the remote branch with yours)

Untracked files:

```
(use "git add <file>..." to include in what will be committed)  
artifacts/  
docs/KHELSUTRA_PRODUCT_OVERVIEW.md
```

```
nothing added to commit but untracked files present (use "git add" to track)  
f5f5e52 feat(eval): R5 ? blob-fallback splits the continuously-active blob (default-OFF) (#203)  
8c68143 feat(eval): R1 ? net over-seg promotion guard unblocks the cap6+R4 milestone (#201)  
2eaccd5 feat(eval): nightly rally-quality regression tripwire (golden corpus, CPU) (#202)  
34b9caa feat(eval): R4 ? rally-END inactivity trim (default-OFF) + R3 cap re-scope finding (#200)  
7e8f88f feat(eval): R2 ? task-faithful tolerance-match rally metric (augments tIoU) (#199)
```

6. assistant (2026-06-17T14:33:07.343Z)

Now let me view the diff against origin/master:

7. user (2026-06-17T14:33:07.840Z)

(Bash completed with no output)

8. assistant (2026-06-17T14:33:09.276Z)

The file might not have changed on this branch. Let me check if the file exists and its current state:

9. user (2026-06-17T14:33:09.457Z)

```
1      """Batch highlight-reel runner ? orchestrates "generate highlights for these videos
2      into this folder" requests with CLEAN, NON-DESTRUCTIVE, resumable execution.
3
4      Why this exists (lessons paid for the hard way):
5      - **Never delete other work.** Reels + per-video run.logs are written to a LOCAL work
dir
6      first, then COPIED into the destination (atomically: temp name -> os.replace). The
7      destination folder is only ever ADDED to ? we never bulk-delete it. (A bulk `rm` of a
8      Google-Drive-synced folder once lost finished reels + logs.)
9      - **Drive-sync safe.** Generating directly inside a Google Drive folder races the sync
10     client (deleted files resurrect, partial writes sync). Generate locally; publish once.
11     - **Idempotent re-runs.** Each video's prior DB segments are cleared before processing
12     (add_video is an UPSERT and does NOT clear children), so a re-run never accumulates
13     duplicate rallies. Belt-and-suspenders with VideoCompiler's overlap-merge (PR #180).
14     - **Resumable & concurrency-friendly.** A video whose reel already exists in the
15     destination is skipped (unless --force); launching a second request never undoes the
16     first's published reels. The Gemini path also isolates its chunk workdir per video
17     (PR for output/chunks/<video_id>) so concurrent videos don't collide.
18
19     Usage:
20     python -m tools.generate_highlights --videos-dir "G:\\...\\June 12 2026" \
21     --out "G:\\...\\June 12 2026\\Highlights" --segmenter gemini [--force]
22     python -m tools.generate_highlights --video A.MP4 --video B.MP4 --out <dir> ...
23     """
24     from __future__ import annotations
25
26     import argparse
27     import logging
28     import os
29     import shutil
30     import sys
31     import traceback
32     from typing import Any, Dict, List, Optional, cast
33
34     from dotenv import load_dotenv
35
36     sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
37
38     from backend.config import load_config
39     from backend.infrastructure.database import Database
40     from backend.pipeline.segmenters.base import segmenter_registry
41     from backend.pipeline.stitcher.compiler import VideoCompiler
42     from backend.results import Failure
43     from backend.utils.hashing import compute_video_id
44
45     logger = logging.getLogger("highlights_runner")
46
47     # Detectors that run in WSL can't read Google Drive / UNC via /mnt -> localize first.
48     _WSL_DETECTORS = {"wasb_hybrid", "tracknet_hybrid", "yolo_hybrid"}
49     _VIDEO_EXTS = (".mp4", ".MP4", ".mov", ".MOV", ".mkv", ".avi")
50
```

```

51
52 def _atomic_publish(local_path: str, dest_path: str) -> None:
53     """Copy local_path -> dest_path so the destination never sees a partial file.
54
55     Copy to a temp sibling, then os.replace (atomic on the same volume). Never deletes
56     anything else in the destination folder.
57     """
58     os.makedirs(os.path.dirname(dest_path), exist_ok=True)
59     tmp = dest_path + ".publishing.tmp"
60     shutil.copy2(local_path, tmp)
61     os.replace(tmp, dest_path)
62
63
64 def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
65                               local_work_dir: str, db_path: str, force: bool = False,
66                               skip_ai_handoff: bool = False, wasb_stream: bool = False)
-> dict:
67     """Generate one highlight reel per input video into out_dir. Returns a summary dict.
68
69     Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never deletes.
70
71     skip_ai_handoff: run the trajectory-hybrid segmenters fully-local (no Gemini
handoff);
72         candidate windows become the rallies. wasb_stream: have the WASB detector decode
the
73         video on the fly instead of dumping ~46k PNGs (output-preserving fast path,
#178).
74     """
75     os.makedirs(out_dir, exist_ok=True)
76     os.makedirs(local_work_dir, exist_ok=True)
77     needs_local = segmenter in _WSL_DETECTORS
78
79     # Load .env (e.g. GEMINI_API_KEY) so the Gemini handoff works when the runner is
invoked
80     # standalone ? backend.main does this, but this module is its own entrypoint.
Without it the
81     # AI segmenter finds no API key and silently produces zero rallies. The fully-local
path
82     # (skip_ai_handoff) needs no key, so a missing .env is harmless there.
83     load_dotenv()
84
85     cfg = load_config()
86     # Per-run overrides (typed config; the segmenter reads these via its dict-compat
.get).
87     # Logged by the segmenter itself once logging is configured below.
88     if skip_ai_handoff:
89         cfg.indexing.skip_ai_handoff = True
90     if wasb_stream:
91         cfg.indexing.wasb.stream_video = True
92     db = Database(db_path)
93     seg_cls = segmenter_registry.get(segmenter)
94     if seg_cls is None:
95         raise SystemExit(f"unknown segmenter '{segmenter}'; available:
{segmenter_registry.list_available()}")
96
97     # Combined batch log (local; published at the end).
98     fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s",
"%H:%M:%S")
99     if not logging.getLogger().handlers:
100         logging.basicConfig(level=logging.INFO,
handlers=[logging.StreamHandler(sys.stdout)])
101         for h in logging.getLogger().handlers:
102             h.setFormatter(fmt)
103     batch_log_local = os.path.join(local_work_dir, "_batch_run.log")
104     batch_fh = logging.FileHandler(batch_log_local, mode="w", encoding="utf-8")

```

```

105     batch_fh.setFormatter(fmt)
106     logging.getLogger().addHandler(batch_fh)
107
108     summary: Dict[str, List[str]] = {"done": [], "skipped": [], "no_rallies": [],
"failed": []}
109     logger.info("highlights runner: %d videos -> %s (segmenter=%s, force=%s)",
110                 len(video_paths), out_dir, segmenter, force)
111
112     for i, src in enumerate(video_paths, 1):
113         name = os.path.basename(src)
114         stem = os.path.splitext(name)[0]
115         reel_name = f"{stem} - Highlights.mp4"
116         dest_reel = os.path.join(out_dir, reel_name)
117         local_reel = os.path.join(local_work_dir, reel_name)
118
119         if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
120             logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths),
name)
121             summary["skipped"].append(name)
122             continue
123         if not os.path.exists(src):
124             logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths),
name)
125             summary["failed"].append(name)
126             continue
127
128         run_log_local = os.path.join(local_work_dir, f"{stem} - run.log")
129         run_fh = logging.FileHandler(run_log_local, mode="w", encoding="utf-8")
130         run_fh.setFormatter(fmt)
131         logging.getLogger().addHandler(run_fh)
132         staged_src: Optional[str] = None
133         try:
134             logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
135             if needs_local:
136                 staged_src = os.path.join(local_work_dir, name)
137                 if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(src)):
138                     logger.info("localizing source (%.0f MB)...", os.path.getsize(src) /
1024 / 1024)
139                     shutil.copy2(src, staged_src)
140                     proc_src = staged_src
141             else:
142                 proc_src = src
143
144             vid = compute_video_id(proc_src)
145             db.clear_video_data(vid)          # idempotent: no cross-run accumulation
146             db.add_video(vid, proc_src, "processed", [])
147             logger.info("video_id=%s segmenting (%s)...", vid, segmenter)
148
149             # AppConfig is consumed by segmenters via its dict-compatible .get shim (same as
150             # main.py); cast keeps the type checker honest about that contract.
151             result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_src)
152             if isinstance(result, Failure):
153                 logger.error("segmentation FAILED: %s", getattr(result, "message",
result))
154                 summary["failed"].append(name)
155                 continue
156
157             segs = db.query_play_segments(vid, {})
158             pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
159             logger.info("rallies detected: %d", len(pairs))
160             if not pairs:
161                 logger.warning("no rallies -> no reel")
162                 summary["no_rallies"].append(name)
163                 continue

```

```

164
165     # Compile locally, then publish atomically into the destination.
166     VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)
167     _atomic_publish(local_reel, dest_reel)
168     try:
169         os.remove(local_reel)
170     except OSError:
171         pass
172     logger.info("PUBLISHED reel: %s (%d rallies)", dest_reel, len(pairs))
173     summary["done"].append(name)
174 except Exception:
175     logger.error("ERROR on %s:\n%s", name, traceback.format_exc())
176     summary["failed"].append(name)
177 finally:
178     logging.getLogger().removeHandler(run_fh)
179     run_fh.close()
180     # publish this video's run.log (additive; never deletes anything in out_dir)
181     try:
182         _atomic_publish(run_log_local, os.path.join(out_dir, f"{stem} -
run.log"))
183     except OSError:
184         pass
185     if staged_src and os.path.exists(staged_src):
186         try:
187             os.remove(staged_src)
188         except OSError:
189             pass
190
191     logging.getLogger().removeHandler(batch_fh)
192     batch_fh.close()
193     try:
194         _atomic_publish(batch_log_local, os.path.join(out_dir, "_batch_run.log"))
195     except OSError:
196         pass
197     logger.info("DONE. %s", {k: len(v) for k, v in summary.items()})
198     return summary
199
200
201 def _resolve_videos(args) -> List[str]:
202     if args.video:
203         return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v
in args.video]
204     if args.videos_dir:
205         return sorted(
206             os.path.join(args.videos_dir, f) for f in os.listdir(args.videos_dir)
207             if f.endswith(_VIDEO_EXTS) and os.path.isfile(os.path.join(args.videos_dir,
f))
208         )
209     raise SystemExit("provide --videos-dir or one/more --video")
210
211
212 def main() -> None:
213     ap = argparse.ArgumentParser(description="Batch highlight-reel runner (clean, non-
destructive).")
214     ap.add_argument("--videos-dir", help="folder of source videos (all videos in it,
unless --video given)")
215     ap.add_argument("--video", action="append", help="explicit source video
(repeatable)")
216     ap.add_argument("--out", required=True, help="destination folder for reels +
run.logs")
217     ap.add_argument("--segmenter", default="gemini", help="segmenter registry key
(default: gemini)")
218     ap.add_argument("--local-work", default=os.path.join("output", "highlights_work"),
219                     help="local staging dir (reels compiled here, then published)")
220     ap.add_argument("--db", default=os.path.join("output", "sports_indexer.db"))

```

```

221         ap.add_argument("--force", action="store_true", help="regenerate even if a reel
already exists")
222         ap.add_argument("--skip-ai-handoff", action="store_true",
223                         help="FULLY-LOCAL: trajectory-hybrid segmenters emit candidate
windows as "
224                             "rallies with NO Gemini handoff (no API key / upload). Ignored
by --segmenter gemini.")
225         ap.add_argument("--wasb-stream", action="store_true",
226                         help="WASB decode-on-the-fly fast path (#178; output-preserving),
instead of "
227                             "dumping ~46k PNGs. Only affects wasb_hybrid.")
228         args = ap.parse_args()
229         generate_highlights_batch(_resolve_videos(args), args.out, segmenter=args.segmenter,
230                                 local_work_dir=args.local_work, db_path=args.db,
force=args.force,
231                                 skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream)
232
233
234     if __name__ == "__main__":
235         main()
236

```

10. assistant (2026-06-17T14:33:10.877Z)

Now let me check the master branch version:

11. user (2026-06-17T14:33:11.395Z)

""Batch highlight-reel runner ? orchestrates "generate highlights for these videos into this folder" requests with CLEAN, NON-DESTRUCTIVE, resumable execution.

Why this exists (lessons paid for the hard way):

- **Never delete other work.** Reels + per-video run.logs are written to a LOCAL work dir first, then COPIED into the destination (atomically: temp name -> os.replace). The destination folder is only ever ADDED to ? we never bulk-delete it. (A bulk `rm` of a Google-Drive-synced folder once lost finished reels + logs.)
- **Drive-sync safe.** Generating directly inside a Google Drive folder races the sync client (deleted files resurrect, partial writes sync). Generate locally; publish once.
- **Idempotent re-runs.** Each video's prior DB segments are cleared before processing (add_video is an UPSERT and does NOT clear children), so a re-run never accumulates duplicate rallies. Belt-and-suspenders with VideoCompiler's overlap-merge (PR #180).
- **Resumable & concurrency-friendly.** A video whose reel already exists in the destination is skipped (unless --force); launching a second request never undoes the first's published reels. The Gemini path also isolates its chunk workdir per video (PR for output/chunks/<video_id>) so concurrent videos don't collide.

Usage:

```

python -m tools.generate_highlights --videos-dir "G:\...\June 12 2026" \
--out "G:\...\June 12 2026\Highlights" --segmenter gemini [--force]
python -m tools.generate_highlights --video A.MP4 --video B.MP4 --out <dir> ...
"""

```

```

from __future__ import annotations

```

```

import argparse
import logging
import os
import shutil
import sys
import traceback
from typing import Any, Dict, List, Optional, cast

```

```

from dotenv import load_dotenv

```

```

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

```

```

from backend.config import load_config
from backend.infrastructure.database import Database
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.stitcher.compiler import VideoCompiler
from backend.results import Failure
from backend.utils.hashing import compute_video_id

```

```

logger = logging.getLogger("highlights_runner")

```

Detectors that run in WSL can't read Google Drive / UNC via /mnt -> localize first.

```

_WSL_DETECTORS = {"wasb_hybrid", "tracknet_hybrid", "yolo_hybrid"}
_VIDEO_EXTS = (".mp4", ".MP4", ".mov", ".MOV", ".mkv", ".avi")

```

```

def _atomic_publish(local_path: str, dest_path: str) -> None:

```

```

    """Copy local_path -> dest_path so the destination never sees a partial file.

```

```

    Copy to a temp sibling, then os.replace (atomic on the same volume). Never deletes
    anything else in the destination folder.

```

```

    """

```

```

    os.makedirs(os.path.dirname(dest_path), exist_ok=True)

```

```

    tmp = dest_path + ".publishing.tmp"

```

```

    shutil.copy2(local_path, tmp)

```

```

    os.replace(tmp, dest_path)

```

```

def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
                              local_work_dir: str, db_path: str, force: bool = False,
                              skip_ai_handoff: bool = False, wasb_stream: bool = False) -> dict:
    """Generate one highlight reel per input video into out_dir. Returns a summary dict.

```

```

    Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never deletes.

```

```

    skip_ai_handoff: run the trajectory-hybrid segmenters fully-local (no Gemini handoff);
    candidate windows become the rallies. wasb_stream: have the WASB detector decode the
    video on the fly instead of dumping ~46k PNGs (output-preserving fast path, #178).

```

```

    """

```

```

    os.makedirs(out_dir, exist_ok=True)

```

```

    os.makedirs(local_work_dir, exist_ok=True)

```

```

    needs_local = segmenter in _WSL_DETECTORS

```

```

    # Load .env (e.g. GEMINI_API_KEY) so the Gemini handoff works when the runner is invoked
    # standalone ? backend.main does this, but this module is its own entrypoint. Without it the
    # AI segmenter finds no API key and silently produces zero rallies. The fully-local path
    # (skip_ai_handoff) needs no key, so a missing .env is harmless there.

```

```

    load_dotenv()

```

```

    cfg = load_config()

```

```

    # Per-run overrides (typed config; the segmenter reads these via its dict-compatible .get).

```

```

    # Logged by the segmenter itself once logging is configured below.

```

```

    if skip_ai_handoff:

```

```

        cfg.indexing.skip_ai_handoff = True

```

```

    if wasb_stream:

```

```

        cfg.indexing.wasb.stream_video = True

```

```

    db = Database(db_path)

```

```

    seg_cls = segmenter_registry.get(segmenter)

```

```

    if seg_cls is None:

```

```

        raise SystemExit(f"unknown segmenter '{segmenter}'; available:

```

```

{segmenter_registry.list_available()}")

```

```

    # Combined batch log (local; published at the end).

```

```

    fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s", "%H:%M:%S")

```

```

    if not logging.getLogger().handlers:

```

```

        logging.basicConfig(level=logging.INFO, handlers=[logging.StreamHandler(sys.stdout)])

```

```

        for h in logging.getLogger().handlers:
            h.setFormatter(fmt)
        batch_log_local = os.path.join(local_work_dir, "_batch_run.log")
        batch_fh = logging.FileHandler(batch_log_local, mode="w", encoding="utf-8")
        batch_fh.setFormatter(fmt)
        logging.getLogger().addHandler(batch_fh)

        summary: Dict[str, List[str]] = {"done": [], "skipped": [], "no_rallies": [], "failed": []}
        logger.info("highlights runner: %d videos -> %s (segmenter=%s, force=%s)",
                    len(video_paths), out_dir, segmenter, force)

    for i, src in enumerate(video_paths, 1):
        name = os.path.basename(src)
        stem = os.path.splitext(name)[0]
        reel_name = f"{stem} - Highlights.mp4"
        dest_reel = os.path.join(out_dir, reel_name)
        local_reel = os.path.join(local_work_dir, reel_name)

        if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
            logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths), name)
            summary["skipped"].append(name)
            continue
        if not os.path.exists(src):
            logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths), name)
            summary["failed"].append(name)
            continue

        run_log_local = os.path.join(local_work_dir, f"{stem} - run.log")
        run_fh = logging.FileHandler(run_log_local, mode="w", encoding="utf-8")
        run_fh.setFormatter(fmt)
        logging.getLogger().addHandler(run_fh)
        staged_src: Optional[str] = None
        try:
            logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
            if needs_local:
                staged_src = os.path.join(local_work_dir, name)
                if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(src)):
                    logger.info("localizing source (%.0f MB)...", os.path.getsize(src) / 1024 /
1024)
                    shutil.copy2(src, staged_src)
                proc_src = staged_src
            else:
                proc_src = src

            vid = compute_video_id(proc_src)
            db.clear_video_data(vid)          # idempotent: no cross-run accumulation
            db.add_video(vid, proc_src, "processed", [])
            logger.info("video_id=%s segmenting (%s)...", vid, segmenter)

            # AppConfig is consumed by segmenters via its dict-compatible .get shim (same as
            # main.py); cast keeps the type checker honest about that contract.
            result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_src)
            if isinstance(result, Failure):
                logger.error("segmentation FAILED: %s", getattr(result, "message", result))
                summary["failed"].append(name)
                continue

            segs = db.query_play_segments(vid, {})
            pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
            logger.info("rallies detected: %d", len(pairs))
            if not pairs:
                logger.warning("no rallies -> no reel")
                summary["no_rallies"].append(name)
                continue

```



```

        # Compile locally, then publish atomically into the destination.
        VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)
        _atomic_publish(local_reel, dest_reel)
    try:
        os.remove(local_reel)
    except OSError:
        pass
    logger.info("PUBLISHED reel: %s (%d rallies)", dest_reel, len(pairs))
    summary["done"].append(name)
except Exception:
    logger.error("ERROR on %s:\n%s", name, traceback.format_exc())
    summary["failed"].append(name)
finally:
    logging.getLogger().removeHandler(run_fh)
    run_fh.close()
    # publish this video's run.log (additive; never deletes anything in out_dir)
    try:
        _atomic_publish(run_log_local, os.path.join(out_dir, f"{stem} - run.log"))
    except OSError:
        pass
    if staged_src and os.path.exists(staged_src):
        try:
            os.remove(staged_src)
        except OSError:
            pass

logging.getLogger().removeHandler(batch_fh)
batch_fh.close()
try:
    _atomic_publish(batch_log_local, os.path.join(out_dir, "_batch_run.log"))
except OSError:
    pass
logger.info("DONE. %s", {k: len(v) for k, v in summary.items()})
return summary

def _resolve_videos(args) -> List[str]:
    if args.video:
        return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v in args.video]
    if args.videos_dir:
        return sorted(
            os.path.join(args.videos_dir, f) for f in os.listdir(args.videos_dir)
            if f.endswith(_VIDEO_EXTS) and os.path.isfile(os.path.join(args.videos_dir, f))
        )
    raise SystemExit("provide --videos-dir or one/more --video")

def main() -> None:
    ap = argparse.ArgumentParser(description="Batch highlight-reel runner (clean, non-destructive).")
    ap.add_argument("--videos-dir", help="folder of source videos (all videos in it, unless --video given)")
    ap.add_argument("--video", action="append", help="explicit source video (repeatable)")
    ap.add_argument("--out", required=True, help="destination folder for reels + run.logs")
    ap.add_argument("--segmenter", default="gemini", help="segmenter registry key (default: gemini)")
    ap.add_argument("--local-work", default=os.path.join("output", "highlights_work"),
                    help="local staging dir (reels compiled here, then published)")
    ap.add_argument("--db", default=os.path.join("output", "sports_indexer.db"))
    ap.add_argument("--force", action="store_true", help="regenerate even if a reel already exists")
    ap.add_argument("--skip-ai-handoff", action="store_true",
                    help="FULLY-LOCAL: trajectory-hybrid segmenters emit candidate windows as "
                         "rallies with NO Gemini handoff (no API key / upload). Ignored by

```

```
--segmenter gemini.")
    ap.add_argument("--wasb-stream", action="store_true",
                    help="WASB decode-on-the-fly fast path (#178; output-preserving), instead of
"
                        "dumping ~46k PNGs. Only affects wasb_hybrid.")
    args = ap.parse_args()
    generate_highlights_batch(_resolve_videos(args), args.out, segmenter=args.segmenter,
                             local_work_dir=args.local_work, db_path=args.db, force=args.force,
                             skip_ai_handoff=args.skip_ai_handoff,
    wasb_stream=args.wasb_stream)

if __name__ == "__main__":
    main()
```

12. assistant (2026-06-17T14:33:12.750Z)

The files are identical. Let me check what branch changes actually exist that might relate to this:

13. user (2026-06-17T14:33:13.555Z)

```
* a6adcc5 fix(runner): honor stitching watermark/padding + add --detect-from for hi-res reels
* b6a39c4 feat(runner): fully-local no-AI segmenter mode (+ WASB stream flag, .env load) (#183)
| * 3ea7ad4 feat(runner): fully-local no-AI segmenter mode (+ WASB stream flag, .env load)
|/
* 4942d51 feat(tools): batch highlight RUNNER ? clean, non-destructive, Drive-safe reel
generation (#182)
* bd3c8b5 feat(tools): batch highlight RUNNER ? clean, non-destructive, Drive-safe reel
generation
```

14. assistant (2026-06-17T14:33:14.994Z)

Aha! There's a commit `a6adcc5` that appears to be newer. Let me check that:

15. user (2026-06-17T14:33:15.406Z)

```
a6adcc57290bd65c3ca7c092d0568ff02cf60378 fix(runner): honor stitching watermark/padding + add
--detect-from for hi-res reels
```

16. assistant (2026-06-17T14:33:16.894Z)

Let me check if this is on the current branch and compare it with master:

17. user (2026-06-17T14:33:17.843Z)

```
f5f5e52 feat(eval): R5 ? blob-fallback splits the continuously-active blob (default-OFF) (#203)
---
a6adcc5 is NOT an ancestor of HEAD
```

18. assistant (2026-06-17T14:33:19.379Z)

So `a6adcc5` is not on this branch. Let me check what the current branch HEAD contains for that file and compare master with it:

19. user (2026-06-17T14:33:19.920Z)

```
""Batch highlight-reel runner ? orchestrates "generate highlights for these videos
into this folder" requests with CLEAN, NON-DESTRUCTIVE, resumable execution.
```

Why this exists (lessons paid for the hard way):

- ****Never delete other work.**** Reels + per-video run.logs are written to a LOCAL work dir first, then COPIED into the destination (atomically: temp name -> os.replace). The

destination folder is only ever ADDED to ? we never bulk-delete it. (A bulk `rm` of a Google-Drive-synced folder once lost finished reels + logs.)

- ****Drive-sync safe.**** Generating directly inside a Google Drive folder races the sync client (deleted files resurrect, partial writes sync). Generate locally; publish once.
- ****Idempotent re-runs.**** Each video's prior DB segments are cleared before processing (add_video is an UPSERT and does NOT clear children), so a re-run never accumulates duplicate rallies. Belt-and-suspenders with VideoCompiler's overlap-merge (PR #180).
- ****Resumable & concurrency-friendly.**** A video whose reel already exists in the destination is skipped (unless --force); launching a second request never undoes the first's published reels. The Gemini path also isolates its chunk workdir per video (PR for output/chunks/<video_id>) so concurrent videos don't collide.

Usage:

```
python -m tools.generate_highlights --videos-dir "G:\\...\\June 12 2026" \
--out "G:\\...\\June 12 2026\\Highlights" --segmenter gemini [--force]
python -m tools.generate_highlights --video A.MP4 --video B.MP4 --out <dir> ...
"""
```

```
from __future__ import annotations
```

```
import argparse
import logging
import os
import shutil
import sys
import traceback
from typing import Any, Dict, List, Optional, cast
```

```
from dotenv import load_dotenv
```

```
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
```

```
from backend.config import load_config
from backend.infrastructure.database import Database
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.stitcher.compiler import VideoCompiler
from backend.results import Failure
from backend.utils.hashing import compute_video_id
```

```
logger = logging.getLogger("highlights_runner")
```

Detectors that run in WSL can't read Google Drive / UNC via /mnt -> localize first.

```
_WSL_DETECTORS = {"wasb_hybrid", "tracknet_hybrid", "yolo_hybrid"}
_VIDEO_EXTS = (".mp4", ".MP4", ".mov", ".MOV", ".mkv", ".avi")
```

```
def _atomic_publish(local_path: str, dest_path: str) -> None:
```

```
    """Copy local_path -> dest_path so the destination never sees a partial file.
```

```
    Copy to a temp sibling, then os.replace (atomic on the same volume). Never deletes
    anything else in the destination folder.
```

```
    """
```

```
    os.makedirs(os.path.dirname(dest_path), exist_ok=True)
    tmp = dest_path + ".publishing.tmp"
    shutil.copy2(local_path, tmp)
    os.replace(tmp, dest_path)
```

```
def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
                              local_work_dir: str, db_path: str, force: bool = False,
                              skip_ai_handoff: bool = False, wasb_stream: bool = False,
                              detect_from_paths: Optional[List[str]] = None) -> dict:
    """Generate one highlight reel per input video into out_dir. Returns a summary dict.
```

```
    Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never deletes.
```

```

skip_ai_handoff: run the trajectory-hybrid segmenters fully-local (no Gemini handoff);
candidate windows become the rallies. wasb_stream: have the WASB detector decode the
video on the fly instead of dumping ~46k PNGs (output-preserving fast path, #178).
detect_from_paths: optional list parallel to video_paths. When given, the detector runs on
detect_from_paths[i] (a cheaper proxy ? e.g. a 1080p downscale of a 4K source) but the
reel is stitched from video_paths[i] at full resolution. Rally timestamps transfer 1:1
(a pure temporal-preserving downscale shares the timeline). None = detect+stitch the
same source (legacy behaviour).
"""
os.makedirs(out_dir, exist_ok=True)
os.makedirs(local_work_dir, exist_ok=True)
needs_local = segmenter in _WSL_DETECTORS

# Load .env (e.g. GEMINI_API_KEY) so the Gemini handoff works when the runner is invoked
# standalone ? backend.main does this, but this module is its own entrypoint. Without it the
# AI segmenter finds no API key and silently produces zero rallies. The fully-local path
# (skip_ai_handoff) needs no key, so a missing .env is harmless there.
load_dotenv()

cfg = load_config()
# Per-run overrides (typed config; the segmenter reads these via its dict-compat .get).
# Logged by the segmenter itself once logging is configured below.
if skip_ai_handoff:
    cfg.indexing.skip_ai_handoff = True
if wasb_stream:
    cfg.indexing.wasb.stream_video = True
db = Database(db_path)
seg_cls = segmenter_registry.get(segmenter)
if seg_cls is None:
    raise SystemExit(f"unknown segmenter '{segmenter}'; available:
{segmenter_registry.list_available()}")

# Combined batch log (local; published at the end).
fmt = logging.Formatter("%(asctime)s [%levelname)s] %(name)s: %(message)s", "%H:%M:%S")
if not logging.getLogger().handlers:
    logging.basicConfig(level=logging.INFO, handlers=[logging.StreamHandler(sys.stdout)])
    for h in logging.getLogger().handlers:
        h.setFormatter(fmt)
batch_log_local = os.path.join(local_work_dir, "_batch_run.log")
batch_fh = logging.FileHandler(batch_log_local, mode="w", encoding="utf-8")
batch_fh.setFormatter(fmt)
logging.getLogger().addHandler(batch_fh)

summary: Dict[str, List[str]] = {"done": [], "skipped": [], "no_rallies": [], "failed": []}
logger.info("highlights runner: %d videos -> %s (segmenter=%s, force=%s)",
            len(video_paths), out_dir, segmenter, force)

for i, src in enumerate(video_paths, 1):
    name = os.path.basename(src)
    stem = os.path.splitext(name)[0]
    reel_name = f"{stem} - Highlights.mp4"
    dest_reel = os.path.join(out_dir, reel_name)
    local_reel = os.path.join(local_work_dir, reel_name)
    # Detection source: a (cheaper) proxy when --detect-from is paired with --video;
    # otherwise the video itself. The reel is ALWAYS stitched from `src` (full res).
    detect_src = detect_from_paths[i - 1] if detect_from_paths else src

    if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
        logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths), name)
        summary["skipped"].append(name)
        continue
    if not os.path.exists(src) or not os.path.exists(detect_src):
        logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s
detect_exists=%s)",
                        i, len(video_paths), name, os.path.exists(src),

```

```

os.path.exists(detect_src))
    summary["failed"].append(name)
    continue

run_log_local = os.path.join(local_work_dir, f"{stem} - run.log")
run_fh = logging.FileHandler(run_log_local, mode="w", encoding="utf-8")
run_fh.setFormatter(fmt)
logging.getLogger().addHandler(run_fh)
staged_src: Optional[str] = None
try:
    logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
    # Localize the DETECTION source for WSL detectors (the WSL FS can't read Drive/UNC
    # via /mnt). The stitch source (`src`) is read by ffmpeg directly during compile, so
    # only the detector input ever needs staging.
    if needs_local:
        staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))
        if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(detect_src)):
            logger.info("localizing detection source (%.0f MB)...",
os.path.getsize(detect_src) / 1024 / 1024)
            shutil.copy2(detect_src, staged_src)
            proc_detect = staged_src
        else:
            proc_detect = detect_src

    vid = compute_video_id(proc_detect)
    db.clear_video_data(vid)          # idempotent: no cross-run accumulation
    db.add_video(vid, proc_detect, "processed", [])
    if detect_src != src:
        logger.info("detecting on proxy '%s' -> stitching reel from '%s'",
            os.path.basename(detect_src), name)
        logger.info("video_id=%s segmenting (%s)...", vid, segmenter)

    # AppConfig is consumed by segmenters via its dict-compatible .get shim (same as
    # main.py); cast keeps the type checker honest about that contract.
    result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_detect)
    if isinstance(result, Failure):
        logger.error("segmentation FAILED: %s", getattr(result, "message", result))
        summary["failed"].append(name)
        continue

    segs = db.query_play_segments(vid, {})
    pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
    logger.info("rallies detected: %d", len(pairs))
    if not pairs:
        logger.warning("no rallies -> no reel")
        summary["no_rallies"].append(name)
        continue

    # Stitch source: the original `src` when detecting on a separate proxy (hi-res
reel);
    # otherwise the (possibly localized) detection input ? identical to legacy
behaviour.

    stitch_src = src if detect_from_paths else proc_detect
    # Honor the configured branding watermark + segment paddings (stitching.*). Read
    # dict-compatibly so a plain-dict config (tests) and the typed AppConfig both work.
    # NOTE: the runner previously dropped these ? reels shipped UN-watermarked with the
    # ctor default paddings; this threads config.json through (the on-by-default mark).
    st = cfg.get("stitching") or {}
    VideoCompiler(
        local_work_dir,
        begin_padding=float(st.get("begin_padding", 0.5)),
        end_padding=float(st.get("end_padding", 0.5)),
        watermark=st.get("watermark"),
    ).compile_highlights(stitch_src, pairs, reel_name)

```

```

    _atomic_publish(local_reel, dest_reel)
try:
    os.remove(local_reel)
except OSError:
    pass
logger.info("PUBLISHED reel: %s (%d rallies)", dest_reel, len(pairs))

```

20. assistant (2026-06-17T14:33:22.310Z)

Now let me see the rest and check the test file as well:

21. user (2026-06-17T14:33:23.038Z)

```

    # ctor default paddings; this threads config.json through (the on-by-default mark).
    st = cfg.get("stitching") or {}
    VideoCompiler(
        local_work_dir,
        begin_padding=float(st.get("begin_padding", 0.5)),
        end_padding=float(st.get("end_padding", 0.5)),
        watermark=st.get("watermark"),
    ).compile_highlights(stitch_src, pairs, reel_name)
    _atomic_publish(local_reel, dest_reel)
try:
    os.remove(local_reel)
except OSError:
    pass
logger.info("PUBLISHED reel: %s (%d rallies)", dest_reel, len(pairs))
summary["done"].append(name)
except Exception:
    logger.error("ERROR on %s:\n%s", name, traceback.format_exc())
    summary["failed"].append(name)
finally:
    logging.getLogger().removeHandler(run_fh)
    run_fh.close()
    # publish this video's run.log (additive; never deletes anything in out_dir)
    try:
        _atomic_publish(run_log_local, os.path.join(out_dir, f"{stem} - run.log"))
    except OSError:
        pass
    if staged_src and os.path.exists(staged_src):
        try:
            os.remove(staged_src)
        except OSError:
            pass

logging.getLogger().removeHandler(batch_fh)
batch_fh.close()
try:
    _atomic_publish(batch_log_local, os.path.join(out_dir, "_batch_run.log"))
except OSError:
    pass
logger.info("DONE. %s", {k: len(v) for k, v in summary.items()})
return summary

def _resolve_videos(args) -> List[str]:
    if args.video:
        return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v in
args.video]
    if args.videos_dir:
        return sorted(
            os.path.join(args.videos_dir, f) for f in os.listdir(args.videos_dir)
            if f.endswith(_VIDEO_EXTS) and os.path.isfile(os.path.join(args.videos_dir, f))
        )
    raise SystemExit("provide --videos-dir or one/more --video")

```

```

def _resolve_detect_from(args, video_paths: List[str]) -> Optional[List[str]]:
    """Resolve --detect-from into a list parallel to video_paths (or None if unused).

    Requires explicit --video (not --videos-dir) and a 1:1 count match ? each detection
    proxy maps to exactly one --video by position."""
    if not getattr(args, "detect_from", None):
        return None
    if not args.video:
        raise SystemExit("--detect-from requires explicit --video (not --videos-dir)")
    if len(args.detect_from) != len(args.video):
        raise SystemExit(f"--detect-from count ({len(args.detect_from)}) must match "
                        f"--video count ({len(args.video)}); pair them 1:1 in order")
    return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v in
            args.detect_from]

def main() -> None:
    ap = argparse.ArgumentParser(description="Batch highlight-reel runner (clean, non-
    destructive).")
    ap.add_argument("--videos-dir", help="folder of source videos (all videos in it, unless
    --video given)")
    ap.add_argument("--video", action="append", help="explicit source video (repeatable)")
    ap.add_argument("--out", required=True, help="destination folder for reels + run.logs")
    ap.add_argument("--segmenter", default="gemini", help="segmenter registry key (default:
    gemini)")
    ap.add_argument("--local-work", default=os.path.join("output", "highlights_work"),
                    help="local staging dir (reels compiled here, then published)")
    ap.add_argument("--db", default=os.path.join("output", "sports_indexer.db"))
    ap.add_argument("--force", action="store_true", help="regenerate even if a reel already
    exists")
    ap.add_argument("--skip-ai-handoff", action="store_true",
                    help="FULLY-LOCAL: trajectory-hybrid segmenters emit candidate windows as "
                    "rallies with NO Gemini handoff (no API key / upload). Ignored by
    --segmenter gemini.")
    ap.add_argument("--wasb-stream", action="store_true",
                    help="WASB decode-on-the-fly fast path (#178; output-preserving), instead of
    "
                    "dumping ~46k PNGs. Only affects wasb_hybrid.")
    ap.add_argument("--detect-from", action="append",
                    help="detection source ? e.g. a 1080p proxy. The detector analyzes THIS "
                    "(cheaper to decode) while the reel is stitched from the matching
    --video "
                    "at full resolution (e.g. the 4K original). Repeat to pair 1:1 with
    --video "
                    "in the same order. Timestamps transfer 1:1 (pure temporal
    downscale).")
    args = ap.parse_args()
    video_paths = _resolve_videos(args)
    detect_from_paths = _resolve_detect_from(args, video_paths)
    generate_highlights_batch(video_paths, args.out, segmenter=args.segmenter,
                             local_work_dir=args.local_work, db_path=args.db, force=args.force,
                             skip_ai_handoff=args.skip_ai_handoff,
    wasb_stream=args.wasb_stream,
                             detect_from_paths=detect_from_paths)

if __name__ == "__main__":
    main()

```

22. assistant (2026-06-17T14:33:24.541Z)

Perfect! Now I have the full picture. Let me check the test file to understand the expected behavior:

23. user (2026-06-17T14:33:25.394Z)

```

"""tools/generate_highlights.py ? the batch highlight runner. The load-bearing guarantees
are NON-DESTRUCTIVE + atomic publish + idempotent skip; those are what we lock here
(heavy deps ? segmenter/compiler/db ? are mocked)."""
import os
from unittest.mock import MagicMock, patch

import pytest

import tools.generate_highlights as gh

def _fake_compiler(captured):
    """A stand-in VideoCompiler that records its ctor kwargs + compile args and writes the
    expected reel file (so the runner's atomic-publish step succeeds)."""
    class FakeCompiler:
        def __init__(self, output_dir, end_padding=1.0, begin_padding=0.5, watermark=None):
            captured["ctor"] = dict(output_dir=output_dir, end_padding=end_padding,
                                    begin_padding=begin_padding, watermark=watermark)

        def compile_highlights(self, raw_video_path, segments, output_filename):
            captured["compile"] = dict(raw=raw_video_path, segments=list(segments),
                                       out=output_filename)
            reel = os.path.join(captured["ctor"]["output_dir"], output_filename)
            with open(reel, "wb") as fh:
                fh.write(b"reel-bytes")
            return reel

    return FakeCompiler

def test_atomic_publish_is_nondestructive(tmp_path):
    src = tmp_path / "local_reel.mp4"
    src.write_bytes(b"reel-bytes")
    dest_dir = tmp_path / "out"
    dest_dir.mkdir()
    sibling = dest_dir / "Someone Else - Highlights.mp4"
    sibling.write_bytes(b"do-not-touch")

    gh._atomic_publish(str(src), str(dest_dir / "Me - Highlights.mp4"))

    assert (dest_dir / "Me - Highlights.mp4").read_bytes() == b"reel-bytes"
    assert sibling.read_bytes() == b"do-not-touch"          # sibling untouched
    assert not list(dest_dir.glob("*.publishing.tmp"))      # no partial left behind

def test_resolve_videos_explicit_and_dir(tmp_path):
    (tmp_path / "a.MP4").write_bytes(b"x")
    (tmp_path / "b.mp4").write_bytes(b"x")
    (tmp_path / "notes.txt").write_text("ignore")
    args = MagicMock(video=None, videos_dir=str(tmp_path))
    found = gh._resolve_videos(args)
    assert [os.path.basename(p) for p in found] == ["a.MP4", "b.mp4"] # videos only, sorted
    args2 = MagicMock(video=["only.MP4"], videos_dir=str(tmp_path))
    assert gh._resolve_videos(args2) == [os.path.join(str(tmp_path), "only.MP4")]

def test_batch_skips_existing_reel_and_preserves_other_work(tmp_path):
    """force=False: a video whose reel already exists is skipped, the segmenter is never
    called, and unrelated files already in the destination are left intact."""
    out = tmp_path / "out"
    out.mkdir()
    (out / "Adarsh v Avi - Highlights.mp4").write_bytes(b"already-published")
    (out / "Adarsh v Avi - run.log").write_text("prior log")
    other = out / "Someone Else - Highlights.mp4"
    other.write_bytes(b"untouched")

```



```

src = tmp_path / "Adarsh v Avi.MP4"
src.write_bytes(b"src")

seg_registry = MagicMock()
with patch.object(gh, "load_config", return_value={}), \
    patch.object(gh, "Database", return_value=MagicMock()), \
    patch.object(gh, "segmenter_registry", seg_registry):
    summary = gh.generate_highlights_batch(
        [str(src)], str(out), segmenter="gemini",
        local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
        force=False)

    assert summary["skipped"] == ["Adarsh v Avi.MP4"]
    # never processed: the resolved segmenter class was never instantiated/run
    seg_registry.get.return_value.assert_not_called()
    assert other.read_bytes() == b"untouched" # other work preserved
    assert (out / "Adarsh v Avi - Highlights.mp4").read_bytes() == b"already-published" # not
    regenerated

def test_batch_applies_local_overrides(tmp_path):
    """--skip-ai-handoff / --wasb-stream inject into the typed config before segmentation."""
    from backend.config import AppConfig
    cfg = AppConfig()
    assert cfg.indexing.skip_ai_handoff is False # default off
    assert cfg.indexing.wasb.stream_video is False

    out = tmp_path / "out"
    out.mkdir()
    src = tmp_path / "clip.MP4"
    src.write_bytes(b"src")

    seg_instance = MagicMock()
    seg_instance.process_video.return_value = [] # no rallies -> no compile/publish
    seg_registry = MagicMock()
    seg_registry.get.return_value = MagicMock(return_value=seg_instance)

    with patch.object(gh, "load_config", return_value=cfg), \
        patch.object(gh, "Database", return_value=MagicMock()), \
        patch.object(gh, "compute_video_id", return_value="vid123"), \
        patch.object(gh, "segmenter_registry", seg_registry):
        gh.generate_highlights_batch(
            [str(src)], str(out), segmenter="gemini",
            local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
            force=True, skip_ai_handoff=True, wasb_stream=True)

    assert cfg.indexing.skip_ai_handoff is True
    assert cfg.indexing.wasb.stream_video is True

def _run_one(tmp_path, *, video_paths, detect_from_paths=None, segmenter="gemini"):
    """Drive generate_highlights_batch once with a fake compiler/segmenter/db, returning the
    captured VideoCompiler ctor+compile args and the compute_video_id mock."""
    from backend.config import AppConfig
    cfg = AppConfig() # watermark on by default
    out = tmp_path / "out"
    out.mkdir()
    db = MagicMock()
    db.query_play_segments.return_value = [{"start_time": 1.0, "end_time": 5.0}]
    seg_instance = MagicMock()
    seg_instance.process_video.return_value = ["ok"] # non-Failure
    seg_registry = MagicMock()
    seg_registry.get.return_value = MagicMock(return_value=seg_instance)
    captured: dict = {}
    cvid = MagicMock(return_value="vid123")

```

```

with patch.object(gh, "load_config", return_value=cfg), \
    patch.object(gh, "Database", return_value=db), \
    patch.object(gh, "compute_video_id", cvid), \
    patch.object(gh, "VideoCompiler", _fake_compiler(captured)), \
    patch.object(gh, "segmenter_registry", seg_registry):
    gh.generate_highlights_batch(
        [str(p) for p in video_paths], str(out), segmenter=segmenter,
        local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
        force=True, detect_from_paths=[str(p) for p in detect_from_paths] if
detect_from_paths else None)
    return captured, cvid, out

def test_batch_threads_watermark_and_padding_into_compiler(tmp_path):
    """REGRESSION: the runner must pass stitching.watermark + begin/end_padding to VideoCompiler
    (it previously constructed VideoCompiler bare -> reels shipped un-watermarked)."""
    src = tmp_path / "clip.MP4"
    src.write_bytes(b"src")
    captured, _cvid, out = _run_one(tmp_path, video_paths=[src])

    wm = captured["ctor"]["watermark"]
    assert wm and wm.get("enabled") is True # on-by-default mark threaded
    through
    assert wm.get("text") == "Generated by Khelsutra.guru"
    assert captured["ctor"]["begin_padding"] == 0.5 # config paddings, not the ctor 1.0
    default
    assert captured["ctor"]["end_padding"] == 0.5
    assert captured["compile"]["raw"] == str(src) # stitches from the (only) source
    assert (out / "clip - Highlights.mp4").exists() # published

def test_batch_detect_from_proxy_stitches_from_original(tmp_path):
    """--detect-from: the detector runs on the proxy (video_id keyed off it) but the reel is
    stitched from the full-res --video, and named after it."""
    orig = tmp_path / "GX01.MP4"
    orig.write_bytes(b"original-4k")
    proxy = tmp_path / "GX01_1080p.mp4"
    proxy.write_bytes(b"proxy-1080")
    captured, cvid, out = _run_one(tmp_path, video_paths=[orig], detect_from_paths=[proxy])

    cvid.assert_called_once_with(str(proxy)) # detection keyed off the proxy
    assert captured["compile"]["raw"] == str(orig) # reel stitched from the original
    assert (out / "GX01 - Highlights.mp4").exists() # named after the original, not the
    proxy

def test_resolve_detect_from_validates_pairing(tmp_path):
    assert gh._resolve_detect_from(MagicMock(detect_from=None), ["v"]) is None
    with pytest.raises(SystemExit): # count mismatch
        gh._resolve_detect_from(MagicMock(detect_from=["a", "b"], video=["v1"],
    videos_dir=None), ["v1"])
    with pytest.raises(SystemExit): # requires explicit --video
        gh._resolve_detect_from(MagicMock(detect_from=["a"], video=None, videos_dir="d"),
    ["v1"])
    abs = os.path.join(os.sep, "abs", "p.mp4")
    assert gh._resolve_detect_from(
        MagicMock(detect_from=[abs], video=["v1"], videos_dir=None), ["v1"]) == [abs]

```

24. assistant (2026-06-17T14:33:27.204Z)

Now let me create a comprehensive analysis. Let me first generate a clean diff:

25. user (2026-06-17T14:33:27.968Z)

```
diff --git a/tools/generate_highlights.py b/tools/generate_highlights.py
```

```

index ae9f045..565663a 100644
--- a/tools/generate_highlights.py
+++ b/tools/generate_highlights.py
@@ -63,7 +63,8 @@ def _atomic_publish(local_path: str, dest_path: str) -> None:

def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
                             local_work_dir: str, db_path: str, force: bool = False,
-                             skip_ai_handoff: bool = False, wasb_stream: bool = False) ->
dict:
+                             skip_ai_handoff: bool = False, wasb_stream: bool = False,
+                             detect_from_paths: Optional[List[str]] = None) -> dict:
    """Generate one highlight reel per input video into out_dir. Returns a summary dict.

    Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never deletes.
@@ -71,6 +72,11 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
    skip_ai_handoff: run the trajectory-hybrid segmenters fully-local (no Gemini handoff);
    candidate windows become the rallies. wasb_stream: have the WASB detector decode the
    video on the fly instead of dumping ~46k PNGs (output-preserving fast path, #178).
+    detect_from_paths: optional list parallel to video_paths. When given, the detector runs on
+    detect_from_paths[i] (a cheaper proxy ? e.g. a 1080p downscale of a 4K source) but the
+    reel is stitched from video_paths[i] at full resolution. Rally timestamps transfer 1:1
+    (a pure temporal-preserving downscale shares the timeline). None = detect+stitch the
+    same source (legacy behaviour).
    """
    os.makedirs(out_dir, exist_ok=True)
    os.makedirs(local_work_dir, exist_ok=True)
@@ -115,13 +121,17 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
    reel_name = f"{stem} - Highlights.mp4"
    dest_reel = os.path.join(out_dir, reel_name)
    local_reel = os.path.join(local_work_dir, reel_name)
+    # Detection source: a (cheaper) proxy when --detect-from is paired with --video;
+    # otherwise the video itself. The reel is ALWAYS stitched from `src` (full res).
+    detect_src = detect_from_paths[i - 1] if detect_from_paths else src

    if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
        logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths), name)
        summary["skipped"].append(name)
        continue
-    if not os.path.exists(src):
-        logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths), name)
+    if not os.path.exists(src) or not os.path.exists(detect_src):
+        logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s
detect_exists=%s)",
+                        i, len(video_paths), name, os.path.exists(src),
os.path.exists(detect_src))
        summary["failed"].append(name)
        continue

@@ -132,23 +142,29 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
    staged_src: Optional[str] = None
    try:
        logger.info("[%d/%d] == %s ==", i, len(video_paths), name)
+        # Localize the DETECTION source for WSL detectors (the WSL FS can't read Drive/UNC
+        # via /mnt). The stitch source (`src`) is read by ffmpeg directly during compile,
so
+        # only the detector input ever needs staging.
+        if needs_local:
-            staged_src = os.path.join(local_work_dir, name)
-            if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(src)):
-                logger.info("localizing source (%.0f MB)...", os.path.getsize(src) / 1024 /
1024)

```

```

-         shutil.copy2(src, staged_src)
-         proc_src = staged_src
+         staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))
+         if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(detect_src)):
+             logger.info("localizing detection source (%.0f MB)...",
os.path.getsize(detect_src) / 1024 / 1024)
+             shutil.copy2(detect_src, staged_src)
+             proc_detect = staged_src
        else:
-         proc_src = src
+         proc_detect = detect_src

-         vid = compute_video_id(proc_src)
+         vid = compute_video_id(proc_detect)
        db.clear_video_data(vid)          # idempotent: no cross-run accumulation
        db.add_video(vid, proc_src, "processed", [])
+         db.add_video(vid, proc_detect, "processed", [])
+         if detect_src != src:
+             logger.info("detecting on proxy '%s' -> stitching reel from '%s'",
+                 os.path.basename(detect_src), name)
+             logger.info("video_id=%s segmenting (%s)...", vid, segmenter)

        # AppConfig is consumed by segmenters via its dict-compat .get shim (same as
        # main.py); cast keeps the type checker honest about that contract.
-         result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_src)
+         result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_detect)
        if isinstance(result, Failure):
            logger.error("segmentation FAILED: %s", getattr(result, "message", result))
            summary["failed"].append(name)
@@ -162,8 +178,20 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter

        summary["no_rallies"].append(name)
        continue

-         # Compile locally, then publish atomically into the destination.
-         VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)
+         # Stitch source: the original `src` when detecting on a separate proxy (hi-res
reel);
+         # otherwise the (possibly localized) detection input ? identical to legacy
behaviour.
+         stitch_src = src if detect_from_paths else proc_detect
+         # Honor the configured branding watermark + segment paddings (stitching.*). Read
+         # dict-compatibly so a plain-dict config (tests) and the typed AppConfig both work.
+         # NOTE: the runner previously dropped these ? reels shipped UN-watermarked with the
+         # ctor default paddings; this threads config.json through (the on-by-default mark).
+         st = cfg.get("stitching") or {}
+         VideoCompiler(
+             local_work_dir,
+             begin_padding=float(st.get("begin_padding", 0.5)),
+             end_padding=float(st.get("end_padding", 0.5)),
+             watermark=st.get("watermark"),
+         ).compile_highlights(stitch_src, pairs, reel_name)
+         _atomic_publish(local_reel, dest_reel)
        try:
            os.remove(local_reel)
@@ -209,6 +237,21 @@ def _resolve_videos(args) -> List[str]:
    raise SystemExit("provide --videos-dir or one/more --video")

+def _resolve_detect_from(args, video_paths: List[str]) -> Optional[List[str]]:
+    """Resolve --detect-from into a list parallel to video_paths (or None if unused).
+
+    Requires explicit --video (not --videos-dir) and a 1:1 count match ? each detection
+    proxy maps to exactly one --video by position."""

```

```

+     if not getattr(args, "detect_from", None):
+         return None
+     if not args.video:
+         raise SystemExit("--detect-from requires explicit --video (not --videos-dir)")
+     if len(args.detect_from) != len(args.video):
+         raise SystemExit(f"--detect-from count ({len(args.detect_from)}) must match "
+                          f"--video count ({len(args.video)}); pair them 1:1 in order")
+     return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v in
args.detect_from]
+
+
def main() -> None:
    ap = argparse.ArgumentParser(description="Batch highlight-reel runner (clean, non-
destructive).")
    ap.add_argument("--videos-dir", help="folder of source videos (all videos in it, unless
--video given)")
@@ -225,10 +268,18 @@ def main() -> None:
    ap.add_argument("--wasb-stream", action="store_true",
                    help="WASB decode-on-the-fly fast path (#178; output-preserving), instead
of "
                        "dumping ~46k PNGs. Only affects wasb_hybrid.")
+    ap.add_argument("--detect-from", action="append",
+                    help="detection source ? e.g. a 1080p proxy. The detector analyzes THIS "
+                    "(cheaper to decode) while the reel is stitched from the matching
--video "
+                    "at full resolution (e.g. the 4K original). Repeat to pair 1:1 with
--video "
+                    "in the same order. Timestamps transfer 1:1 (pure temporal
downscale).")
    args = ap.parse_args()
-    generate_highlights_batch(_resolve_videos(args), args.out, segmenter=args.segmenter,
+    video_paths = _resolve_videos(args)
+    detect_from_paths = _resolve_detect_from(args, video_paths)
+    generate_highlights_batch(video_paths, args.out, segmenter=args.segmenter,
                             local_work_dir=args.local_work, db_path=args.db,
force=args.force,
-                                skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream)
+                                skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream,
+                                detect_from_paths=detect_from_paths)

if __name__ == "__main__":

```

26. assistant (2026-06-17T14:33:29.815Z)

Perfect! Now let me carefully analyze the legacy behavior. Let me trace through what happens when `detect\_from\_paths` is None:

27. user (2026-06-17T14:33:40.043Z)

ANALYSIS: Behavior Preservation When detect_from_paths = None

The task is to verify that when detect_from_paths is None (the default/legacy path), the renamed variables preserve BYTE-EQUIVALENT behavior to the pre-diff code.

LEGACY CODE PATH (when detect_from_paths would be None):

=====

Line 112: for i, src in enumerate(video_paths, 1):

- src = source video path

Line 125-142: if needs_local / else:

- Original: staged_src = os.path.join(local_work_dir, name)

- Original: shutil.copy2(src, staged_src)

- Original: `proc_src = staged_src OR proc_src = src`

```
Line 144: vid = compute_video_id(proc_src)
Line 145: db.add_video(vid, proc_src, "processed", [])
Line 151: result = seg_cls(...).process_video(vid, proc_src)
Line 166: VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)
```

NEW CODE WITH `detect_from_paths=None`:

=====

```
Line 124: detect_src = detect_from_paths[i-1] if detect_from_paths else src
        ? When None: detect_src = src ? CORRECT
```

Line 134-151: `if needs_local / else:`

- New: `staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))`
? `os.path.basename(detect_src)` when `detect_src=src` equals `os.path.basename(src)`
? `os.path.basename(src) = name` (from line 113)
? NEW: `staged_src = os.path.join(local_work_dir, name)` ? SAME

- New: `shutil.copy2(detect_src, staged_src)`
? When `detect_src=src`: `shutil.copy2(src, staged_src)` ? SAME

- New: `proc_detect = staged_src OR proc_detect = detect_src`
? When `detect_src=src`: `proc_detect = staged_src OR proc_detect = src`
? This is EXACTLY the same as `proc_src` ? SAME

```
Line 156: vid = compute_video_id(proc_detect)
        ? proc_detect = proc_src in legacy path ? SAME
```

```
Line 157-158: db.add_video(vid, proc_detect, ...)
        ? proc_detect = proc_src in legacy path ? SAME
```

```
Line 161: result = seg_cls(...).process_video(vid, proc_detect)
        ? proc_detect = proc_src in legacy path ? SAME
```

```
Line 182: stitch_src = src if detect_from_paths else proc_detect
        ? When detect_from_paths=None: stitch_src = proc_detect ? CORRECT
        ? proc_detect = proc_src in legacy path
        ? So: stitch_src = proc_src ? SAME
```

```
Line 189-196: VideoCompiler(...).compile_highlights(stitch_src, ...)
        ? NEW SIGNATURE CHANGE (watermark + padding args added)
        ? This is a BEHAVIOR CHANGE (not a regression per se, but differs from master)
```

CRITICAL DIFFERENCES FROM MASTER:

=====

1. ****VideoCompiler constructor call****:

- MASTER: `VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)`
- NEW: `VideoCompiler(local_work_dir, begin_padding=..., end_padding=..., watermark=...).compile_highlights(stitch_src, pairs, reel_name)`

This adds three new keyword arguments to `VideoCompiler.__init__` that did NOT exist in the master version.

2. ****stitch_src variable introduction****:

- Legacy: used `proc_src` directly for stitching
- New: assigns to `stitch_src = proc_detect` when `detect_from_paths=None`
- These are equivalent BUT the extra variable assignment is added logic

3. ****Naming changes****:

- `proc_src` ? `proc_detect` (in the context of `detect_from_paths` feature)
- The logic flow for `proc_detect` is IDENTICAL to `proc_src` when `detect_from_paths=None`

4. ****Logging changes****:

- NEW: `logger.info("detecting on proxy '%s' -> stitching reel from '%s'")`
- This is only logged when `detect_src != src`, so in legacy path (`detect_src=src`) it's skipped

?

5. ****Config extraction for stitching****:

- NEW: st = cfg.get("stitching") or {}
- Then: begin_padding=float(st.get("begin_padding", 0.5)), etc.
- OLD: VideoCompiler(local_work_dir) used default ctor params (1.0 end_padding, 0.5 begin_padding)
- This is the WATERMARK regression mentioned in the test comment

28. assistant (2026-06-17T14:33:41.404Z)

Now let me check the VideoCompiler signature to understand the actual behavior change:

29. user (2026-06-17T14:33:42.405Z)

```
class VideoCompiler:
    def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float = 0.5,
                  watermark: Optional[Any] = None):
        self.output_dir = output_dir
        self.end_padding = end_padding
        self.begin_padding = begin_padding
        # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
        # normalized to a dict (or None) so this module stays config-package-agnostic.
        self.watermark = self._normalize_watermark(watermark)
        os.makedirs(output_dir, exist_ok=True)

    @staticmethod
    def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
        if watermark is None:
            return None
        if hasattr(watermark, "model_dump"):
            return watermark.model_dump()
        if isinstance(watermark, dict):
            return dict(watermark)
        return None
```

30. assistant (2026-06-17T14:33:44.262Z)

Good! The VideoCompiler already supports these parameters in master. Now let me check if the new version in a6adcc5 passes the same default values:

31. user (2026-06-17T14:33:45.033Z)

```
class VideoCompiler:
    def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float = 0.5,
                  watermark: Optional[Any] = None):
        self.output_dir = output_dir
        self.end_padding = end_padding
        self.begin_padding = begin_padding
        # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
        # normalized to a dict (or None) so this module stays config-package-agnostic.
        self.watermark = self._normalize_watermark(watermark)
        os.makedirs(output_dir, exist_ok=True)

    @staticmethod
    def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
        if watermark is None:
            return None
        if hasattr(watermark, "model_dump"):
            return watermark.model_dump()
        if isinstance(watermark, dict):
            return dict(watermark)
        return None
```

32. assistant (2026-06-17T14:33:46.666Z)

Now let me carefully compare the exact defaults being passed. Let me check what the config default is:

33. user (2026-06-17T14:33:47.419Z)

fatal: path 'backend/config.py' does not exist in 'origin/master'

34. assistant (2026-06-17T14:33:50.713Z)

Let me find the config file:

35. user (2026-06-17T14:33:51.257Z)

(Bash completed with no output)

36. assistant (2026-06-17T14:33:53.180Z)

Let me search for where stitching config is defined:

37. user (2026-06-17T14:33:53.566Z)

```
{
  "sport": "badminton",
  "ai_provider": "gemini",
  "ai_model": "gemini-flash-latest",
  "validation": {
    "min_resolution_width": 1280,
    "min_resolution_height": 720,
    "min_fps": 29.9,
    "min_blur_threshold": 20.0,
    "max_scene_cuts_per_minute": 0.5,
    "relevance": {
      "court_green_lower": [
        35,
        40,
        40
      ],
      "court_green_upper": [
        85,
        255,
        255
      ],
      "court_pixels_min_ratio": 0.05
    }
  },
  "indexing": {
    "default_segmenter": "gemini",
    "use_gpu": true,
    "motion_threshold": 0.08,
    "min_segment_duration": 3.0,
    "dead_time_merge_gap": 2.0,
    "chunk_duration": 90,
    "chunk_overlap": 10,
    "detector_model": "fasterrcnn_resnet50_fpn",
    "detector_score_threshold": 0.5,
    "yolo_velocity_threshold": 0.15,
    "yolo_frame_skip": 3,
    "yolo_tracker": "bytetrack.yaml",
    "yolo_prefetch_workers": 2,
    "min_action_window_duration": 2.0,
    "log_yolo_candidates": true,
    "store_yolo_candidates": true,
  }
}
```



```

"ai_handoff_padding": 2.0,
"ai_max_workers": 1,
"tracknet": {
    "wsl_distro": "Ubuntu",
    "repo_dir": "~/models/TrackNetV4",
    "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
    "conda_env": "TrackNetV4",
    "weights_path": "",
    "queue_length": 5,

```

38. assistant (2026-06-17T14:33:55.048Z)

Let me check the actual config file used in the a6adcc5 commit:

39. user (2026-06-17T14:33:55.445Z)

```

"stitching": {
    "begin_padding": 0.5,
    "end_padding": 0.5,
    "use_cache": false,
    "min_shot_count": 1,
    "watermark": {
        "enabled": true,
        "text": "Generated by Khelsutra.guru",
        "position": "bottom-right",
        "opacity": 0.85,
        "logo_path": "",

```

40. assistant (2026-06-17T14:33:58.252Z)

Perfect! Now I can analyze the actual behavior change. The key insight:

```

- **Old config.json (master)**: No "stitching" section defined ? VideoCompiler gets
`watermark=None` ? defaults to NO watermark
- **New code (a6adcc5)**: Extracts `st = cfg.get("stitching") or {}` ? calls
`st.get("watermark")` ? if config has it, passes it
- **New config.json (a6adcc5)**: Has `"watermark": { "enabled": true, "text": "..." }` ? passes
this to VideoCompiler

```

Let me verify the old config doesn't have watermark:

41. user (2026-06-17T14:33:58.533Z)

```

"stitching": {
    "begin_padding": 0.5,
    "end_padding": 0.5,
    "use_cache": false,
    "min_shot_count": 1,
    "watermark": {
        "enabled": true,
        "text": "Generated by Khelsutra.guru",
        "position": "bottom-right",
        "opacity": 0.85,
        "logo_path": "",

```

42. assistant (2026-06-17T14:34:03.646Z)

Interesting! So both have watermark. But the critical insight is how the runner uses them. Let me trace through exactly what values are passed:

```

**MASTER (origin/master):**
...
VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)
...

```

This implicitly uses defaults: `end\_padding=1.0, begin\_padding=0.5, watermark=None`

```
**NEW (a6adcc5) with detect_from_paths=None:**
```

```
...  
st = cfg.get("stitching") or {}  
VideoCompiler(  
    local_work_dir,  
    begin_padding=float(st.get("begin_padding", 0.5)),  
    end_padding=float(st.get("end_padding", 0.5)),  
    watermark=st.get("watermark"),  
)  
.compile_highlights(stitch_src, pairs, reel_name)  
...
```

So when `cfg.get("stitching")` returns the dict from config.json:

- `begin\_padding = float(st.get("begin\_padding", 0.5))` = `0.5` ?
- `end\_padding = float(st.get("end\_padding", 0.5))` = `0.5` (BUT was `1.0` before!) ? REGRESSION
- `watermark = st.get("watermark")` = the watermark dict (BUT was `None` before!) ? REGRESSION

This is a ****BEHAVIOR REGRESSION****. The old code used `VideoCompiler(local\_work\_dir)` which has `end\_padding=1.0` as the default, but the new code extracts `end\_padding=0.5` from config, changing the actual reels generated.

Let me verify this finding by checking the actual VideoCompiler defaults:

43. user (2026-06-17T14:34:04.256Z)

```
def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float = 0.5,  
             self.end_padding = end_padding  
             logger.info(f"Trimming {len(segments)} segments  
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
```

44. assistant (2026-06-17T14:34:05.764Z)

Perfect. Now let me document all findings:

45. user (2026-06-17T14:34:16.097Z)

Structured output provided successfully